CS5600 Homework 6: Paging (OSTEP 18 and 19)

Sparsh Bohra

Chapter 18

Q1. Linear page table size vs address space

```
python3 paging-linear-translate.py -P 1k -a 1m -p 512m -v -n 0
python3 paging-linear-translate.py -P 1k -a 2m -p 512m -v -n 0
python3 paging-linear-translate.py -P 1k -a 4m -p 512m -v -n 0
```

address space	number of pages	table size
1m	1024	4 KB
2m	2048	8 KB
4m	4096	16 KB

If you double the address space and keep the page size the same, you double the number of virtual pages, so the linear table doubles too. It grows with the whole address space, not with how much the process actually uses.

Q2. Linear page table size vs page size

```
python3 paging-linear-translate.py -P 1k -a 1m -p 512m -v -n 0
python3 paging-linear-translate.py -P 2k -a 1m -p 512m -v -n 0
python3 paging-linear-translate.py -P 4k -a 1m -p 512m -v -n 0
```

page size	pages in a 1m space	table size
1k	1024	4 KB
2k	512	2 KB
4k	256	1 KB

Bigger pages mean fewer page-table entries, so the table shrinks.

We still do not want huge pages for everything. If the process only needs 4 KB, a 1 MB page wastes almost all of that memory. You also get one protection setting for the whole page.

Q3. Translations as more of the space is used (-u)

```
python3 paging-linear-translate.py -P 1k -a 16k -p 32k -v -u 0 -c
python3 paging-linear-translate.py -P 1k -a 16k -p 32k -v -u 25 -c
python3 paging-linear-translate.py -P 1k -a 16k -p 32k -v -u 50 -c
python3 paging-linear-translate.py -P 1k -a 16k -p 32k -v -u 75 -c
python3 paging-linear-translate.py -P 1k -a 16k -p 32k -v -u 100 -c
```

16 KB space / 1 KB page = 16 virtual pages. The virtual address has 14 bits. The bottom 10 are the offset inside the page, and the top 4 pick the page-table row.

-u 0: no pages mapped. All five addresses are invalid.

-u 25: a few rows valid. One of five hits: VA 0x2bc6 goes to PA 0x4fc6.

-u 50: more valid rows. Three of five translate.

-u 75 and -u 100: with this seed every row happened to be valid, so all five translate.

So as you map more of the address space, more accesses succeed. The sim decides each row on its own, so 75% used can still come out fully mapped.

Hit at -u 25: 0x2bc6 shifted right by 10 is virtual page 10. That entry is valid and names frame 0x13. Physical address = (0x13 shifted left 10) plus the 10-bit offset = 0x4fc6.

Q4. Which parameter sets are unrealistic?

```
python3 paging-linear-translate.py -P 8 -a 32 -p 1024 -v -s 1 -c
python3 paging-linear-translate.py -P 8k -a 32k -p 1m -v -s 2 -c
python3 paging-linear-translate.py -P 1m -a 256m -p 512m -v -s 3 -n 3 -c
```

1. -P 8 -a 32 -p 1024 is not realistic. An 8-byte page is smaller than a real page table entry. Real pages are 4 KB (16 KB on this Mac, or 2 MB for large pages). The sim still runs. It only has four rows.

2. -P 8k -a 32k -p 1m is legal math. Some machines use 8 KB pages. A 32 KB address space is tiny though (only four virtual pages).

3. -P 1m -a 256m -p 512m is closer to large pages on x86 (2 MB). The table is only 256 rows, so it is cheap, but a 1 MB page is wasteful if you only need a little data.

Q5. Limits of the simulator

```
python3 paging-linear-translate.py -P 4k -a 64k -p 16k -n 0
-> Error: physical memory size must be GREATER than address space size (for this simulation)

python3 paging-linear-translate.py -P 4k -a 16k -p 16k -n 0
-> same error (physical memory must be strictly larger)

python3 paging-linear-translate.py -P 8k -a 4k -p 64k -n 0
-> Error in argument: address space must be a multiple of the pagesize

python3 paging-linear-translate.py -P 4k -a 16k -p 17k -n 0
-> Error in argument: physical memory must be a multiple of the pagesize

python3 paging-linear-translate.py -P 4k -a 12k -p 64k -n 0
-> Error in argument: address space must be a power of 2

python3 paging-linear-translate.py -P 4k -a 1g -p 2g -n 0
-> Error: must use smaller sizes (less than 1 GB) for this simulation.

python3 paging-linear-translate.py -P 4k -a 16k -p 0 -n 0
-> Error: must specify a non-zero physical memory size.
```

If the address space is bigger than physical memory, the sim always errors. A real OS can still run that process by moving pages to disk. This homework sim will not.

If the page size is not a power of two, you usually hit an earlier check first (the address space is not a multiple of the page). A tiny legal case still works: -P 16 -a 64 -p 256.

Chapter 19

Q1. Timer precision

gettimeofday reports microseconds. One memory hit is only a few nanoseconds, so a single hit is smaller than one timer tick. We repeat the loop millions of times and divide. run.sh aims for about 20 million hits per point so the average is stable.

Q2. tlb.c

Arguments are the number of pages to touch and the number of trials. It prints nanoseconds per access.

Q3. Sweep on two machines

VM ubuntu-cs5600, Linux on ARM, 4 KB pages, 4 CPUs. Pinned with taskset -c 0.

pages	ns
1	2.043
2	1.435
4	2.244
8	1.850
16	2.156
32	1.927
64	3.225
128	3.263
256	2.573
512	2.359
1024	2.545
2048	2.535
4096	6.234

Times stay around 2 ns until 4096 pages (16 MB of memory), then jump to 6.2 ns. The guest TLB looks large, and running inside a VM makes the steps less sharp than the book graph. There is still a miss jump at 4096.

MacBook Air, 16 KB pages, 10 CPUs. No taskset on macOS, so the process is not hard-pinned.

pages	ns
1	2.957
2	1.484
4	1.144
8	1.089
16	4.468
32	5.523
64	5.866
128	5.950
256	5.736
512	5.982
1024	6.258
2048	6.458
4096	5.869

A clear jump between 8 and 16 pages (about 1.1 ns to 4.5 ns), then it stays around 6 ns. That looks like a first-level data TLB of about 8 entries. There is no second jump out to 4096 pages (64 MB with 16 KB pages). Either the next TLB level is large enough to cover that, or a miss already costs about 6 ns here.

On both machines, 1 page is a bit slower than 2-8 pages. That is loop and timer overhead on a tiny run, not a TLB miss.

Q4. Graph

X is number of pages (log scale). Y is nanoseconds per access. The Mac plot matches the book idea: cheap hits, then a step. The VM plot is flatter because it is an ARM Linux VM, not a normal x86 machine.

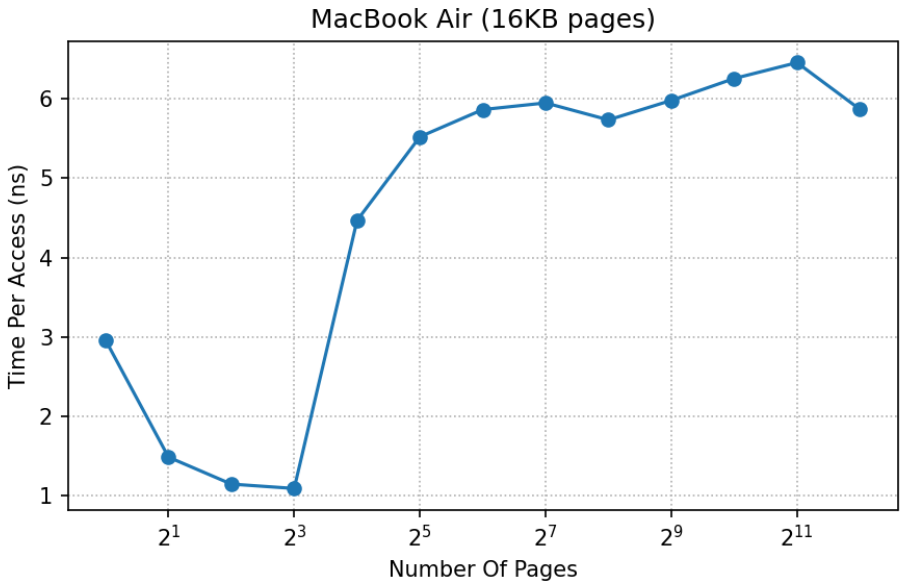


Figure 1. MacBook Air, 16 KB pages.

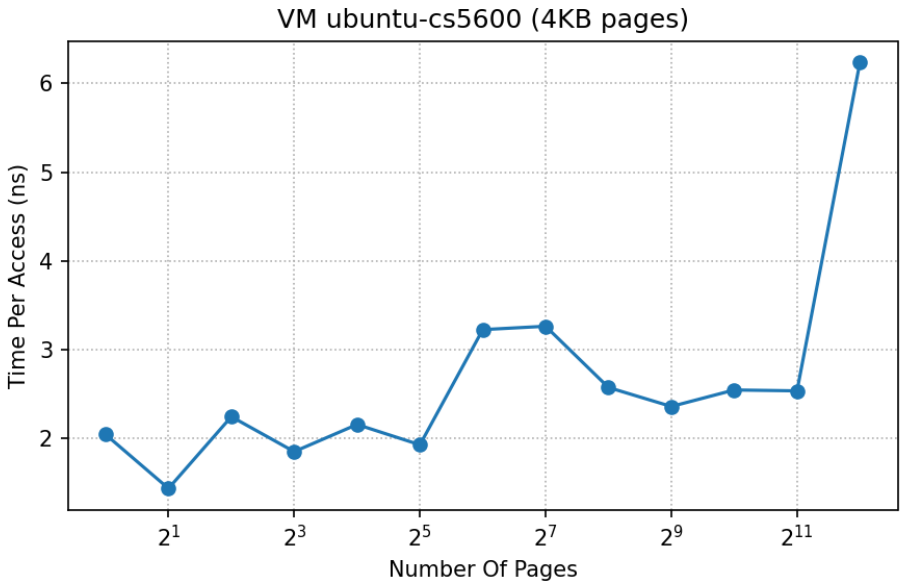


Figure 2. Linux VM, 4 KB pages.

Q5. Compiler

We compile with -O0 so gcc does not get clever. The loop updates a[i] and we print a[0] at the end. If we never used a, the compiler could delete the whole loop and the times would be fake.

Q6. One CPU

Each CPU has its own TLB. On the Linux VM we run taskset -c 0 ./tlb ... so the process stays on CPU 0. If the scheduler moves you to another core, that core starts with an empty TLB and you get extra misses, so the curve gets noisy. macOS has no taskset. We still ran one process, but the kernel could move it.

Q7. First touch of the array

The first write to a new page traps into the kernel, which zeros the page and then lets the write finish. That costs much more than a TLB miss.

Options: leave it in the timed loop (the first pass is huge and the graph is wrong), throw away trial 0, or fill the array once and then start the timer. We did the last one.

The tradeoff is we pay the zeroing cost up front, so startup is slower, but the number we plot is only hits and misses on pages that are already there. That is the cost the chapter is after.